UC San Diego

DSC 102
Systems for Scalable Analytics

Alexander Kolovos

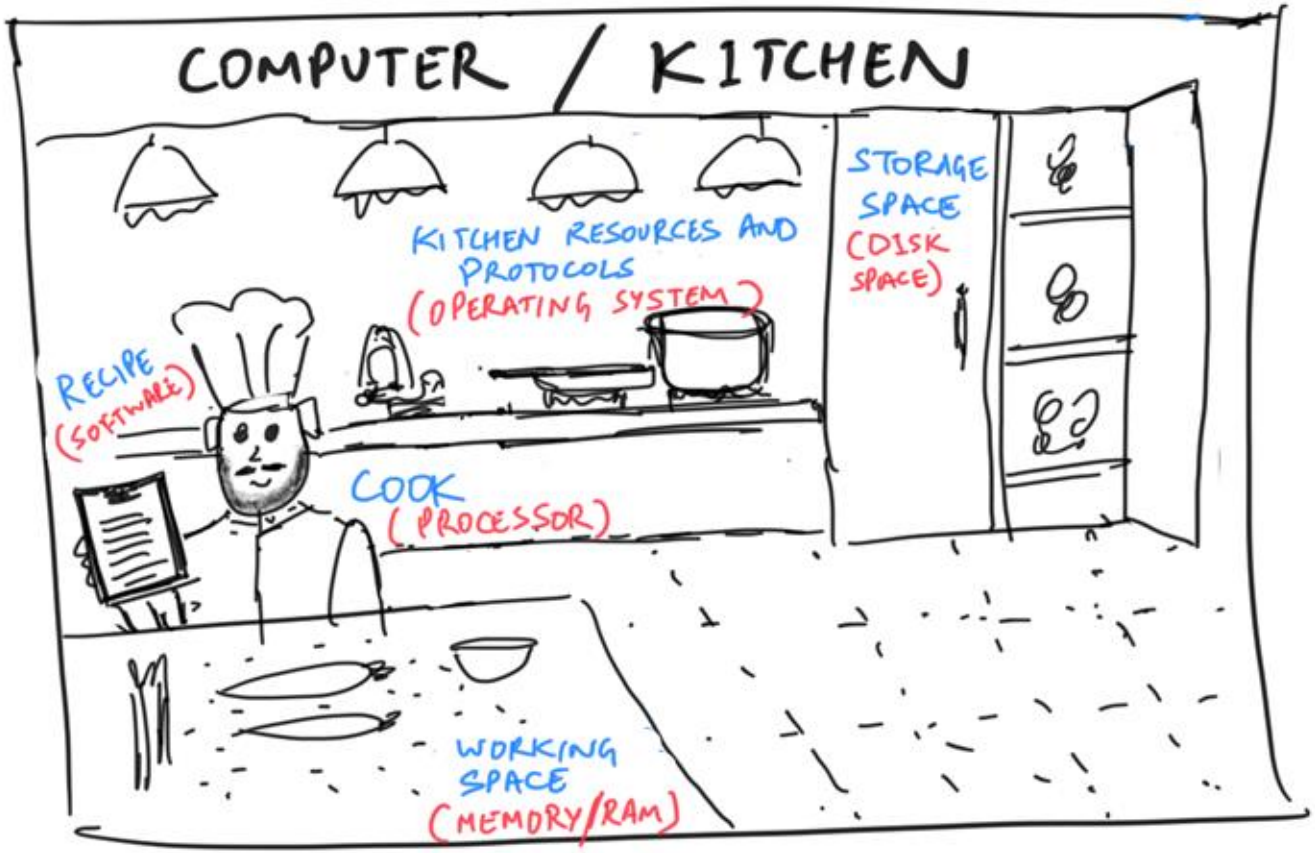
Topic 1: Basics of Machine Resources
Part 2: Operating Systems

Ch. 2, 4.1-4.2, 6, 7, 13, 14.1, 18.1, 21, 22, 26, 36, 37, 39, and 40.1-40.2 of Comet Book

Topic 1 – Outline

- Basics of Computer Organization
  - Digital Representation of Data
  - Processors and Memory Hierarchy
- Basics of Operating Systems (OS)
  - Process Management: Virtualization; Concurrency
  - Filesystem and Data Files
  - Main Memory Management
- Persistent Data Storage

What Is An OS? Why Do We Need It?



It provides a ***secure*** way of accessing ***resources*** to get useful work done

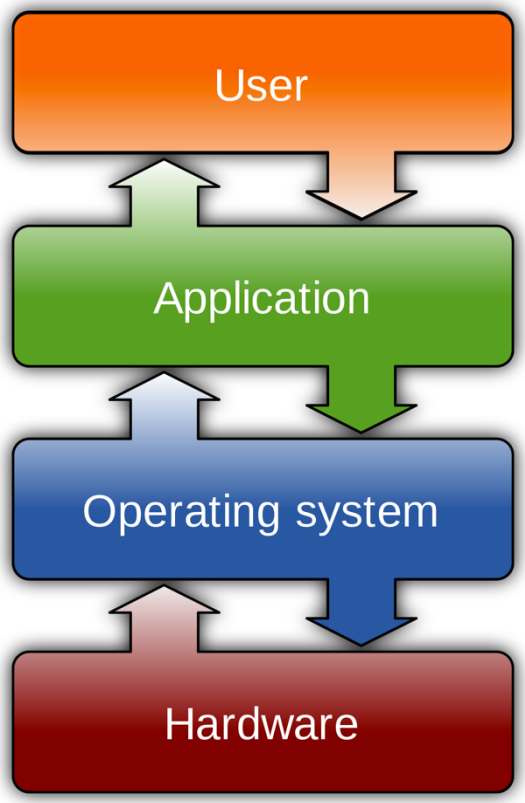
Hmm... What Does It Do, Precisely?

1. Virtualizes ***Hardware resources and manages contention***:
   - CPUs
   - Memory
   - GPUs
   - Storage
   - Network
2. Provides tools to handle ***Concurrency***
3. ***Secures access*** to the hardware
4. Allows users to ***interface with devices*** (USB drives, speakers, microphones, cameras etc.)

What Form Does It Take?

- An OS is a **large set of interrelated programs** that *make it easier* for applications and user-written programs to use computer hardware *effectively*, *efficiently*, and *securely*
  - Analogue example: Consider the government's role in a country.
- Without OS, computer users must speak machine code!
- 2 key principles in OS (any system) design & implementation:
  - **Modularity:** Divide system into *functionally cohesive components* that each do their jobs well
    - Gov't example: Consider executive-legislature-judiciary split
  - **Abstraction:** *Layers of functionalities* from low-level (close to hardware) to high level (close to user)
    - Gov't example: Consider local-city-county-state-federal levels.

Role of an OS in a Computer



APIs/Interface of Application Software

APIs of OS aka “System Calls”

Hardware-specific code parts of OS

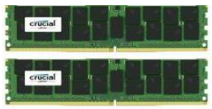
“Application Software” notion is now more complex due to multiple tiers of abstraction; “Platform Software” or “Software Framework” is a new tier between “Application” and OS

Understanding An OS: The Trifecta

- Abstractions : The What
- Mechanisms : The How
- Policies – Orchestration : The conductor

Key Components of OS

- **Kernel:** The core of an OS with modules to abstract the hardware and APIs for programs to use
- Auxiliary parts of OS include shell/terminal, file browser for usability, extra programs installed by I/O devices, etc.

	"System Call" APIs				
Kernel Components	Process Management	Main Memory Management	Filesystems	Networking	Device Drivers
Functionality	*Virtualize* processor; "**Process**" abstraction; **Concurrency**	*Virtualize* Main Memory	*Virtualize* disks; "**File**" abstraction; **Persistence**	Commun. over network	Talk to other I/O devices
	Hardware device-specific programs				
Hardware					

Topic 1 – Outline

- Basics of Computer Organization
  - Digital Representation of Data
  - Processors and Memory Hierarchy
- Basics of Operating Systems (OS)
  - Process Management: Virtualization; Concurrency
  - Filesystem and Data Files
  - Main Memory Management
- Persistent Data Storage

Sharing The Hardware

You are a chef in a kitchen with exactly one stove (the CPU).
You have one physical copy of a 'Signature Chocolate Cake' recipe.
Suddenly, three different waiters rush in: Tables 5, 12, and 18 have all ordered that exact cake at the same time.

1. Since there is only one recipe, are there three 'cakes' or just one? How do we define the difference between the **recipe** on the paper and the **act of making** those three specific cakes?

2. If the chef has to stop working on Table 5's cake to start Table 12's, what specific things must the chef 'save' or 'write down' so they don't accidentally put salt in Table 5's batter when they return to it?

The Abstraction of a Process

- **Process:** A *running* program, the central abstraction in OS
  - Started by OS when a program is executed by user
  - OS keeps inventory of "alive" processes (**Process List**) and handles apportioning of hardware among processes

Q: *Why bother knowing process management in Data Science?*

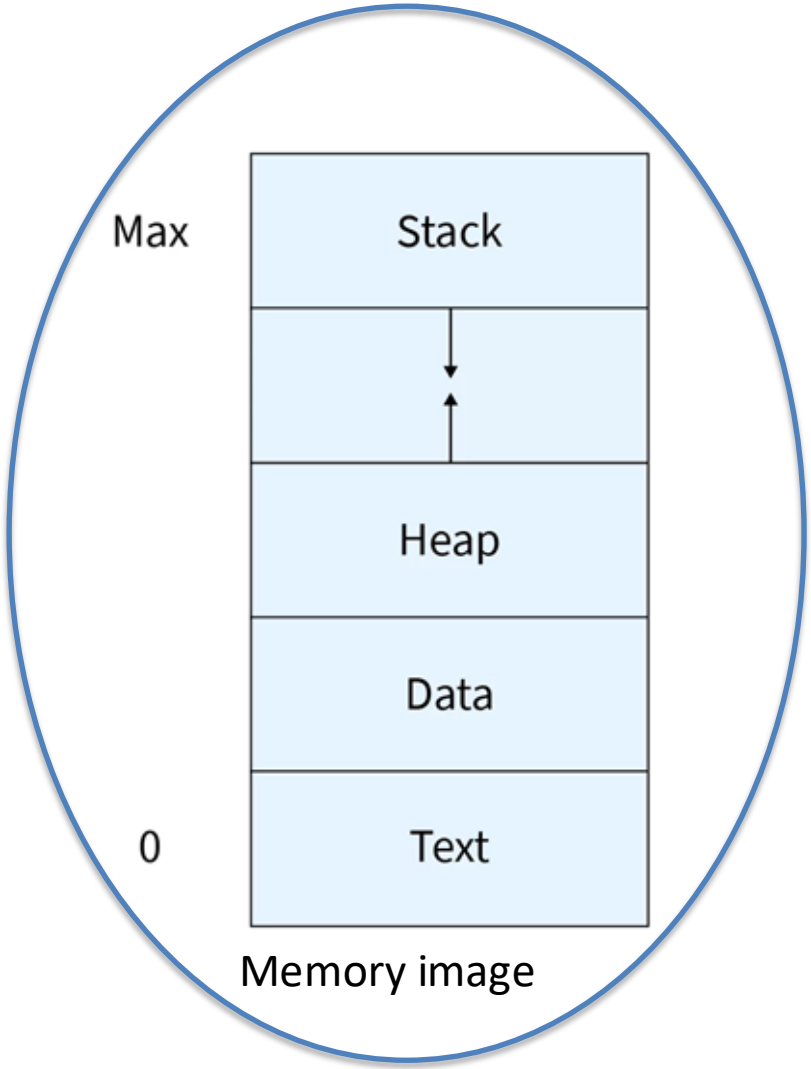
- A *query* is a program that becomes a process
- A data system typically *abstracts away* process management because user specifies the queries / processes in system's API

DASK

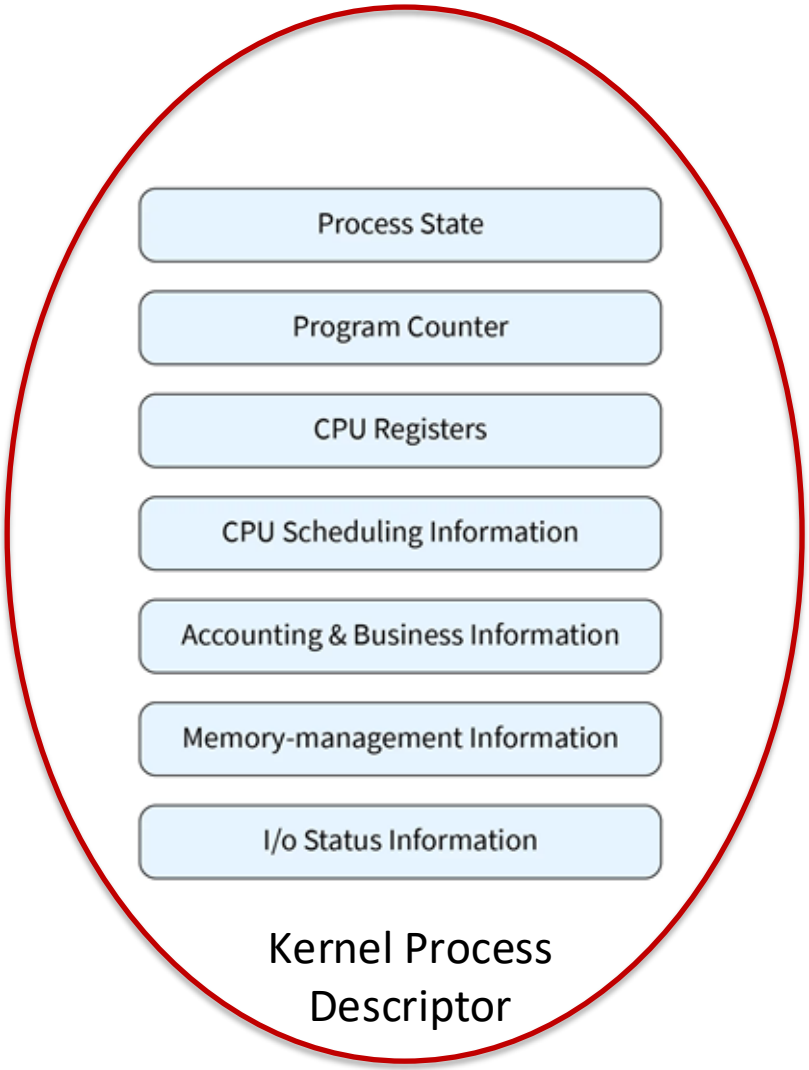
APACHE Spark™

- But in the cloud era, things are up in the air! Will help to know a bit how data-intensive computations are handled under the hood.

Machine State of a Process



+

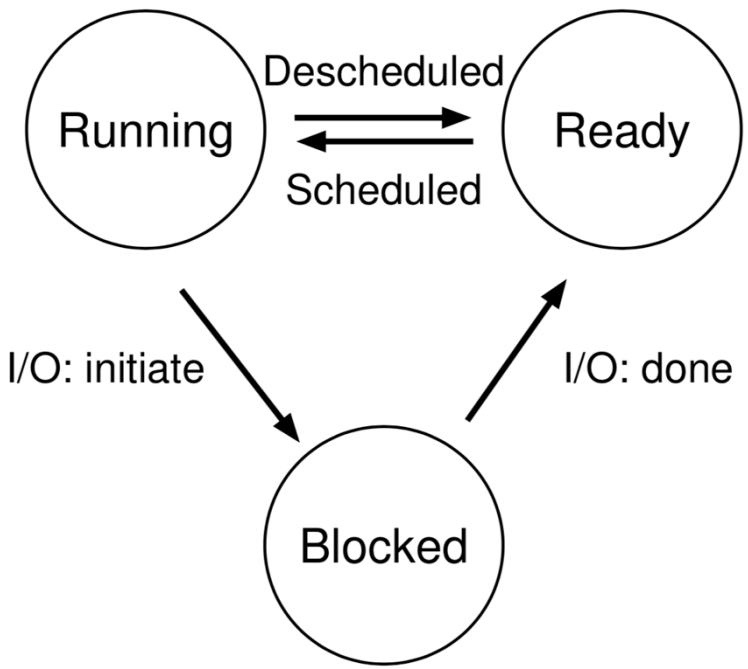


Process Vs. Program

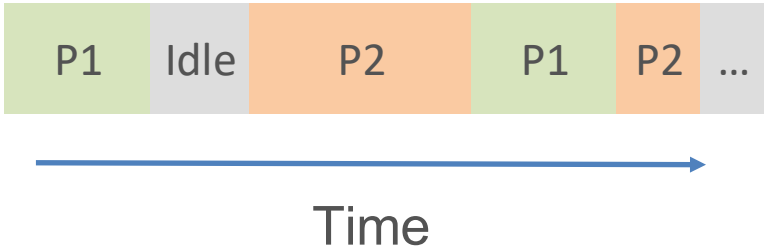
Feature	Process	Program
Definition	An instance of a program in execution.	A set of instructions that can be executed by a computer.
Existence	Dynamic entity that exists for a limited time.	Static entity that exists until it is deleted.
Creation	Created when a program is executed.	Created when a program is written or compiled.
Resources	Requires resources such as CPU time, memory, and I/O devices.	Does not require any resources.
State	Can be in one of several states, such as new, runnable, running, blocked, or terminated.	Always in the same state.
Control block	Has a process control block (PCB) that contains information about the process.	Does not have a control block.
Execution	Can be executed concurrently with other processes.	Can only be executed one at a time.
Example	A web browser, a word processor, a game.	A source code file, an executable file.

Process Management by OS

- A Process can be in one of 3 states:



- Gantt Chart: A visualization: Shows what process runs when (on processor)



How does a process start?

- With a program given to it, OS does the following:
  1. **Create** a process (get Process ID; add to Process List)
  2. Assign part of DRAM to process, aka its **Address Space**
  3. Load code and static data (if applicable) to that space
  4. Set up the inputs needed to run program's *main()*
  5. Update process' **State** to *Ready*
  6. When process is **scheduled** (*Running*), OS temporarily hands off control to process to run the show!
  7. Eventually, process finishes or run **Destroy**

Process API: Interfacing With The OS

- CREATE – fork()
  - Creates an identical copy of the process as a child
- WAIT – wait()
  - Waits for a process to complete execution
- EXECUTE – execvp*()
  - Starts a program programmatically!
- STOP – kill()
  - Sends a signal to a process

An Example - 1

```
import os

# Create a child process
# using os.fork() method
pid = os.fork()

# pid greater than 0 represents
# the parent process
if pid > 0 :
    print("I am parent process:")
    print("Process ID:", os.getpid())
    print("Child's process ID:", pid)

# pid equal to 0 represents
# the created child process
else :
    print("\nI am child process:")
    print("Process ID:", os.getpid())
    print("Parent's process ID:", os.getppid())

# If any error occurred while
# using os.fork() method
# OSError will be raised
```

```
I am Parent process
Process ID: 10793
Child's process ID: 10794
```

```
I am child process
Process ID: 10794
Parent's process ID: 10793
```

An Example - 2

```
import sys
import os
import time

sys.stdout.write("Ready to fork? (Press enter to continue)")

sys.stdout.flush()
sys.stdin.readline()

os.fork()
print("Hi there")
time.sleep(3)
print("Howdy")
```

The Hostile Guest Chef

To make the kitchen faster, you allow 'Guest Chefs' (User Programs) to come in and cook their own meals directly on your stove. This is much faster than you doing it for them. However, one guest decides they like the kitchen so much they are **never leaving**. They ignore you when you ask for the stove back, and they even try to change the master locks on the pantry.

1. **The Control Problem:** If the Guest Chef is currently 'running' the stove, and you (the OS) are standing in the corner, how do you actually stop them? If they have the 'CPU,' do you even have the power to speak?

2. **The Hardware Solution:** Imagine the stove has a 'User Mode' and a 'Master Mode.' What are three things a Guest Chef should *never* be allowed to do in User Mode? (e.g., *Turning off fire alarms, locking other chefs out*).

3. **The Timer:** Suppose we install a 'Kitchen Timer' that the Guest Chef cannot touch. Every 10 minutes, it rings and **automatically** freezes the guest, handing the spatula back to you. Is this 'fair' to the guest? Is it 'safe' for the kitchen?

Requirements On Process Execution

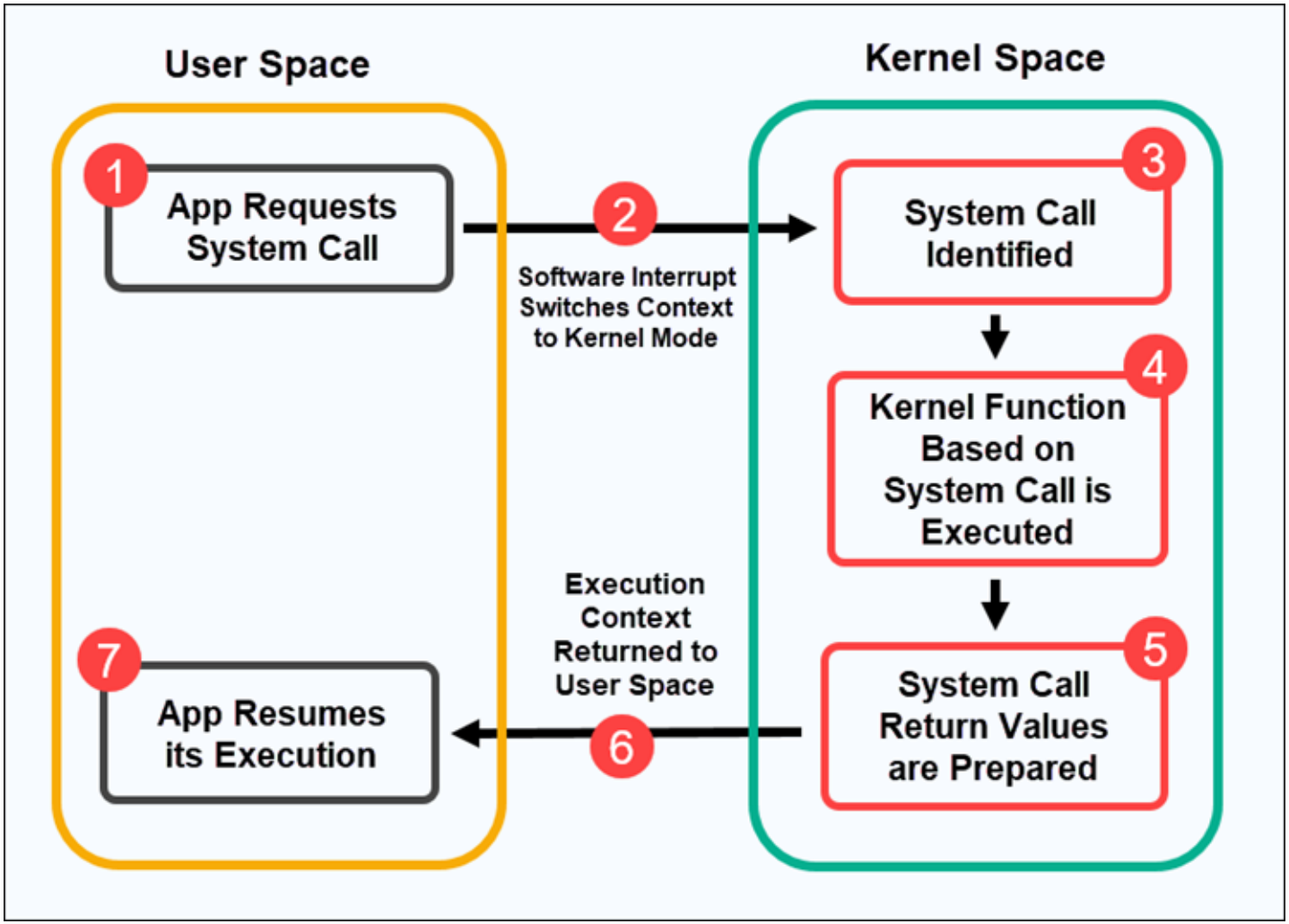
1. **The need for speed**
   - don't want to use an intermediary to execute user code
   - **Direct execution**

     Q: But is it not risky/foolish for OS to hand off control of hardware to a process (random user-written program)?!
   - "**Limited Direct Execution**" - Restricted Operations

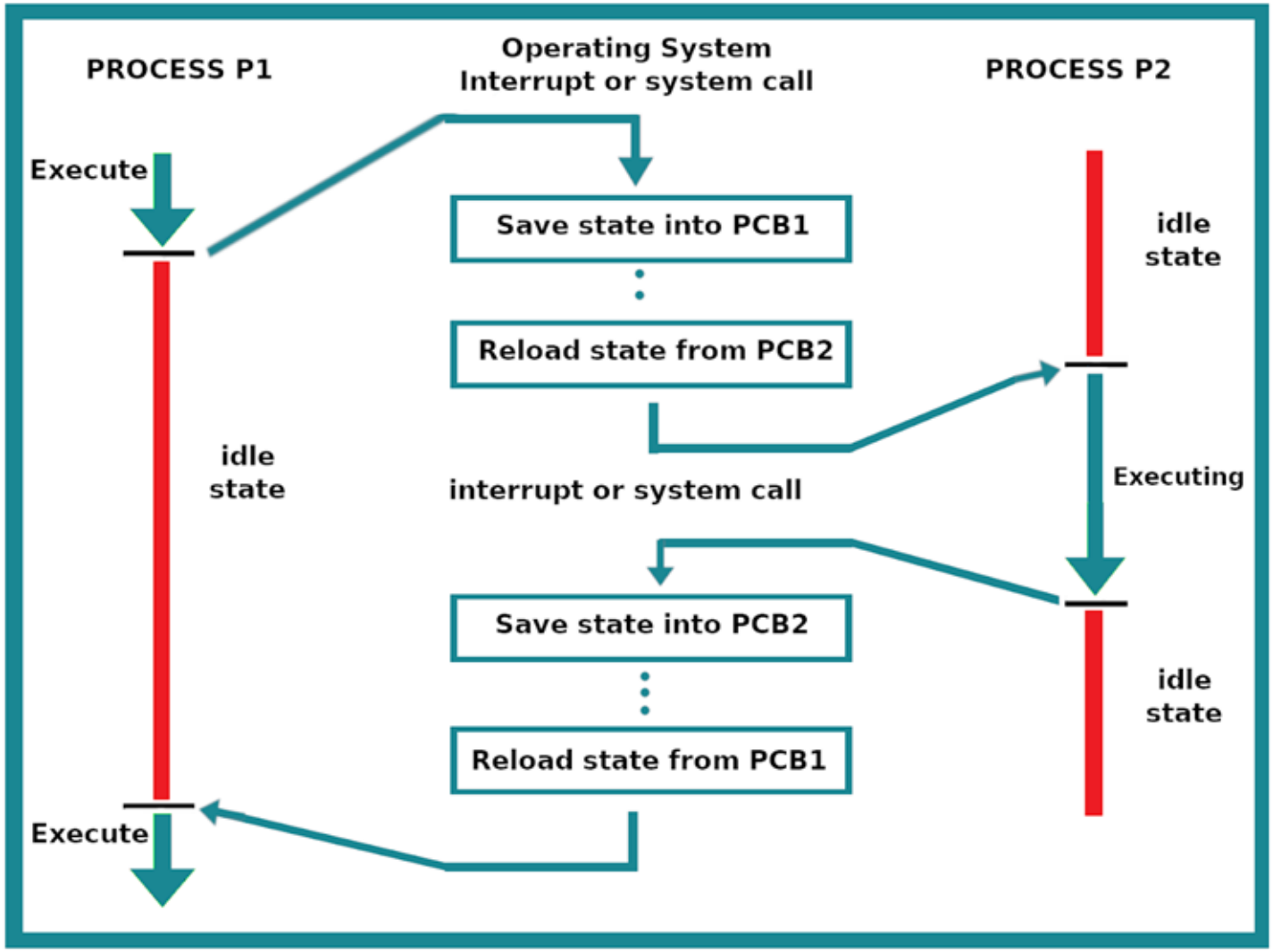
2. **Fairness**
   - How does it ensure that every process gets a chance to use the CPI
   - Once it has handed over control of the processor to a process how does the OS get it back?
   - **Context Switch & Timer interrupts**

3. **Isolation**
   - How does it ensure that one process cannot interfere with another?

System Call Mechanism ("Limited")



Context Switch Mechanism



The "Angry Customer" Dilemma

It’s the Saturday night rush. You have three orders on the counter:

- **Order A:** A 2-minute Appetizer.
- **Order B:** A 60-minute Prime Rib.
- **Order C:** A 3-minute Soup. They arrived in the order: **B, then A, then C**

1. **Ordered Approach:** If you strictly follow the 'First-In, First-Out' rule, you start the 60-minute Prime Rib first. How does the customer waiting for the 2-minute appetizer feel? How do you fix this?
2. **The Efficiency Approach:** If you decide to do the quicker dishes first, the total 'average wait time' for all three tables drops significantly. But what happens if 100 more '2-minute appetizers' keep arriving all night? Does the Prime Rib ever get cooked?
3. **The Compromise:** What if you spend 1 minute on the Prime Rib, then 1 minute on the Appetizer, then 1 minute on the Soup, and rotate? Does the Prime Rib take *longer* or *shorter* to finish this way? Who benefits most from this 'choppy' style of cooking?

Scheduling Policies/Algorithms

- **Schedule:** Record of what process runs on each CPU & when
- Policy controls how OS time-shares CPUs among processes
- Key terms for a process (aka **job**):
  - **Arrival Time**: Time when process gets created
  - **Job Length**: Duration of time needed for process
  - **Start Time**: Times when process first starts on processor
  - **Completion Time**: Time when process finishes/killed
  - **Response** Time = [Start Time] – [Arrival Time]
  - **Turnaround** Time = [Completion Time] – [Arrival Time]
- **Workload:** Set of processes, arrival times, and job lengths that OS Scheduler has to handle

Scheduling Policies/Algorithms

- In general, OS may not know all Arrival Times and Job Lengths beforehand! But **preemption** is possible (act of the scheduler temporarily interrupting a running task so that another runs first.)

- **Key Principle:** Inherent tension in scheduling between overall workload *performance* and allocation *fairness*
  - Performance metric is usually *Average Turnaround Time*
  - Fairness: Many metrics exist (e.g., Jain's fairness index)

Scheduling Policies/Algorithms

- 100s of scheduling policies studied!

  We will be overviewing some well-known ones:

  FIFO (First-In-First-Out, or **FCFS**; First-Come-First-Served)

  SJF (Shortest Job First)

  SCTF (Shortest Completion Time First)

  Round Robin

  Random, etc.
- Different criteria for ranking; preemptive vs not
- Complex “multi-level feedback queue” schedulers
- ML-based schedulers are “hot” nowadays!

Scheduling Metrics

$T_{turnaround} = T_{completion} - T_{arrival}$

$T_{response} = T_{firstrun} - T_{arrival}$

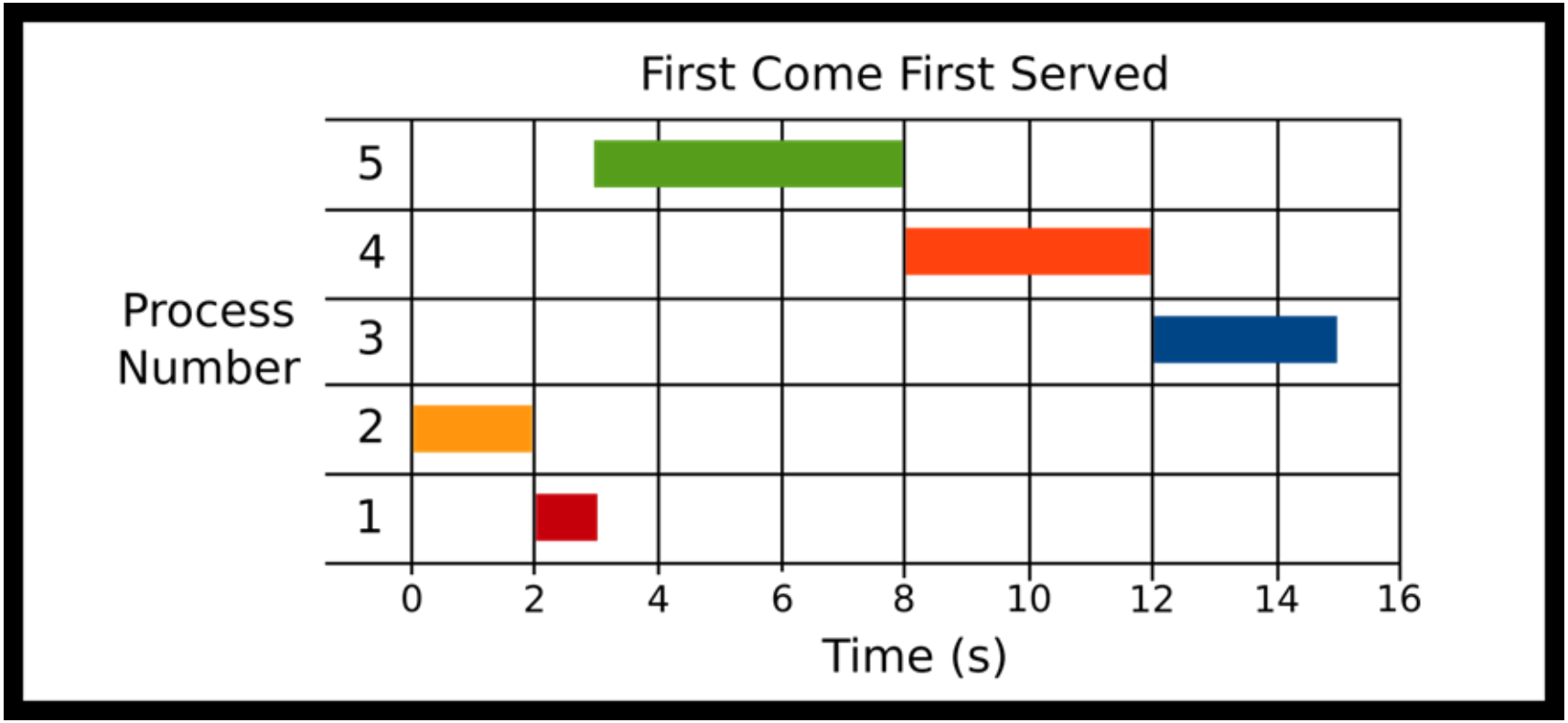
For Interactive jobs

FCFS

P2 at 0ms
P1 at 1000ms
P5 at 3000ms
P4 at 4000ms
P3 at 5000ms

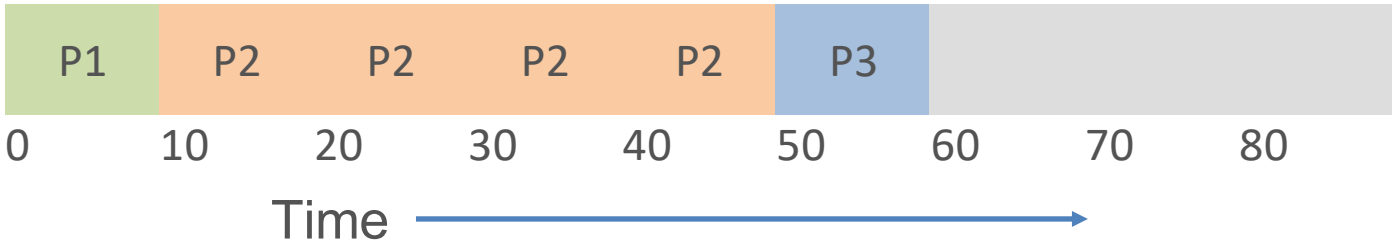
Average turnaround time = (2+1+5+4+3)/5 = 3 seconds

Average response = 0 seconds!



Where FCFS Fails

Example: P1, P2, P3
of lengths 10,40,10 units, respectively,
arrive closely in that order



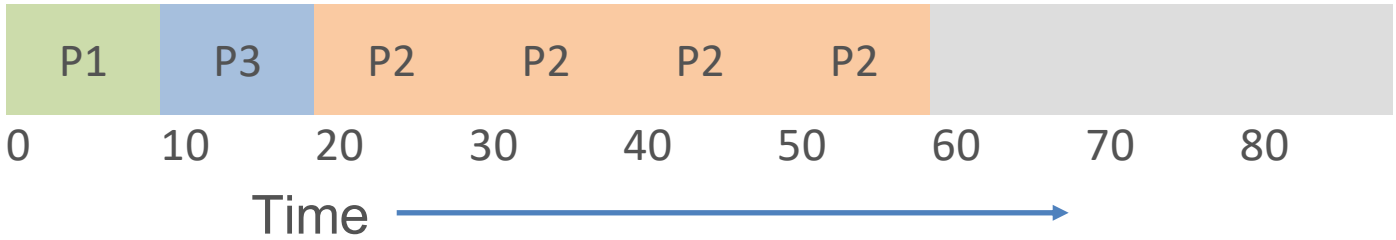
Process	Arrival t1	Start t2	Completion t3	Response t4 = t2-t1	Turnaround t5 = t3-t1
P1	0	0	10	0	10
P2	0	10	50	10	50
P3	0	50	60	50	60
			Avg:	20	40

Main con: Short jobs may wait a lot, aka "**Convoy Effect**"

SJF

- Shortest Job First
- Ranking by Job Length; no preemption

Example: P1, P2, P3
of lengths 10,40,10 units, respectively
arrive closely in that order



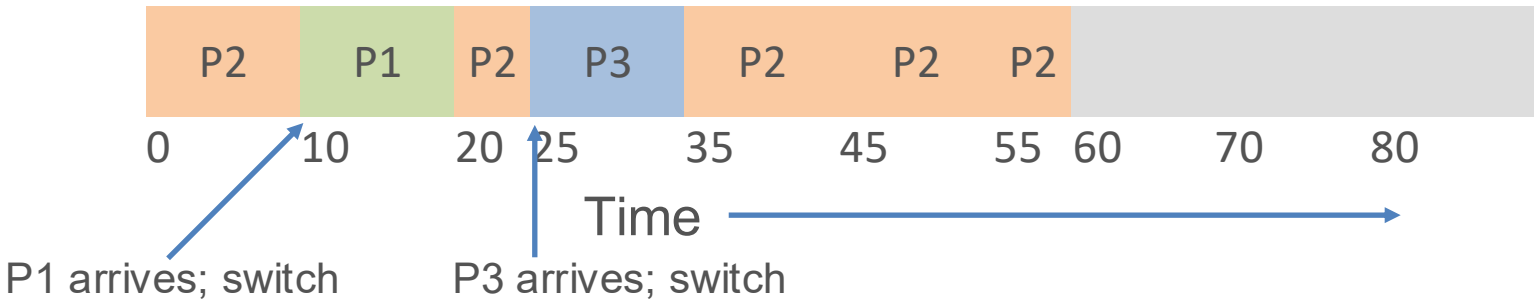
Process	Arrival t1	Start t2	Completion t3	Response t4 = t2-t1	Turnaround t5 = t3-t1
P1	0	0	10	0	10
P2	0	20	60	20	60
P3	0	10	20	10	20
			(FCFS Avg: 20 and 40)	Avg: 10	30

Main con: Need to know all job sizes beforehand.

SCTF

- Shortest Completion Time First (aka SJF with preemption)
- Ranking by Job Length; preemption applied (arrival time indifferent)

Example: P1, P2, P3 of lengths 10,40,10 units arrive at different times



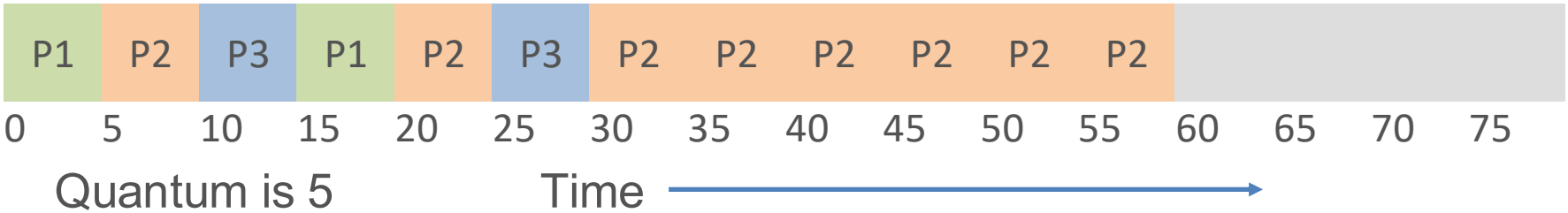
Process	Arrival t1	Start t2	Completion t3	Response t4 = t2-t1	Turnaround t5 = t3-t1
P1	10	10	20	0	10
P2	0	0	60	0	60
P3	25	25	35	0	10
			(SJF Avg: 10 and 30) Avg:	0	26.7

Main cons: Job lengths might be unknown beforehand;
possibly extensive completion times for long jobs (starvation)

Round Robin

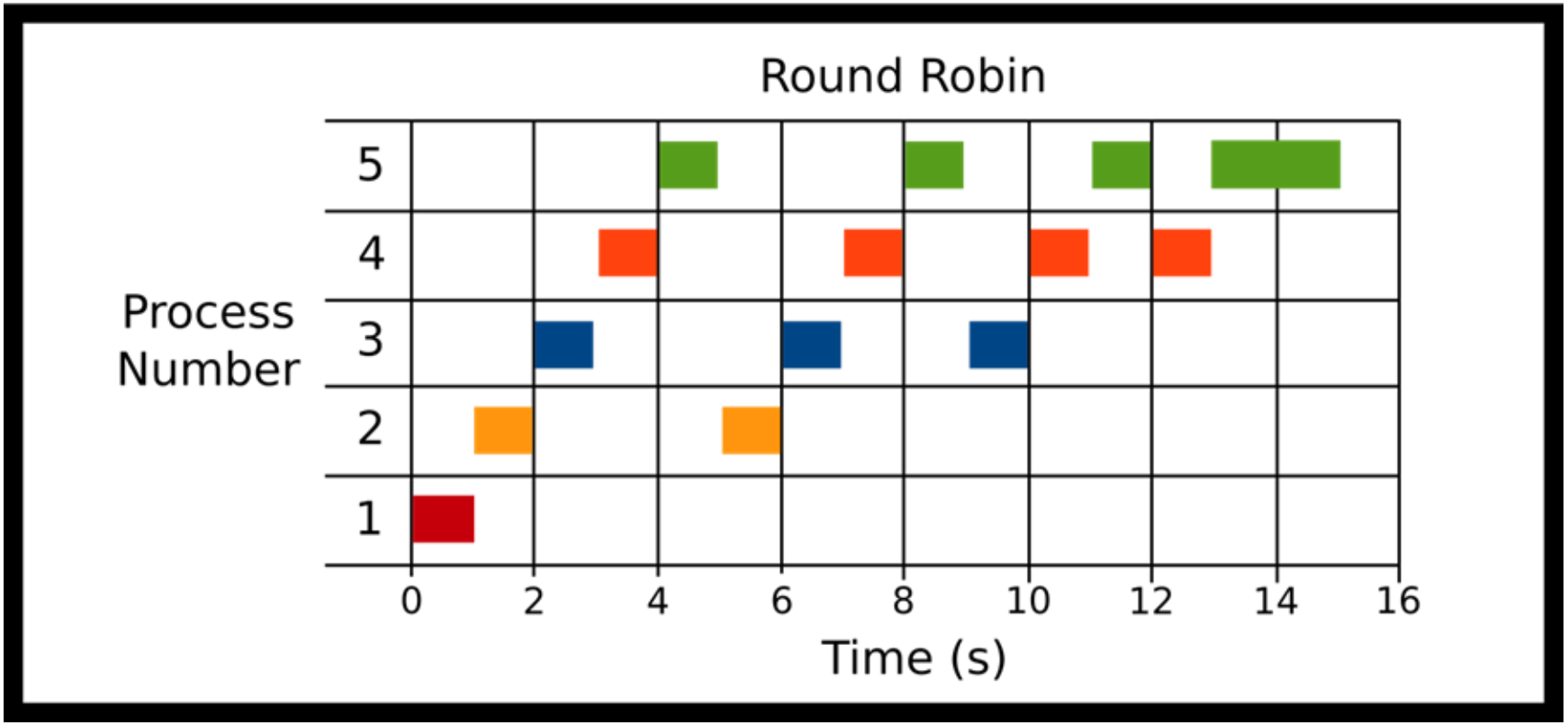
- In Round Robin job lengths need not be known
- Fixed time *quantum* given to each job; cycle through jobs

Example: P1, P2, P3
of lengths 10,40,10 units, respectively,
arrive closely in that order



Process	Arrival t1	Start t2	Completion t3	Response t4 = t2-t1	Turnaround t5 = t3-t1
P1	0	0	20	0	20
P2	0	5	60	5	60
P3	0	10	30	10	30
(SJF Avg: 10 & 30; SCTF Avg: 0 & 26.7)				Avg: 5	36.7

Round Robin - Analysis



Fairness?
Turnaround time?

Response Time?
What else????

Separate Restaurants Vs. Busy Diner

Imagine your town has two ways to serve food:

- **Process Diner:** Two entirely separate catering trucks parked on opposite sides of town. Each truck has its own stove, its own fridge, its own ingredients, and its own head chef.

- **Threaded Diner:** One large, busy diner. Inside this single building, there are three different chefs all working at the same time, sharing the same walk-in fridge, the same pantry, and the same stovetop.

Scenarios

Scenario: In the **Process Model**, the chef in Truck A accidentally leaves the burner on and the truck catches fire. What happens to the food in Truck B?
Scenario: In the **Thread Model**, one of the three chefs accidentally spills a giant bucket of salt into the shared soup pot that everyone is using. What happens to the orders the other two chefs are working on?

The Question: If safety and 'not crashing the whole system' is our priority, which model is better? Why?

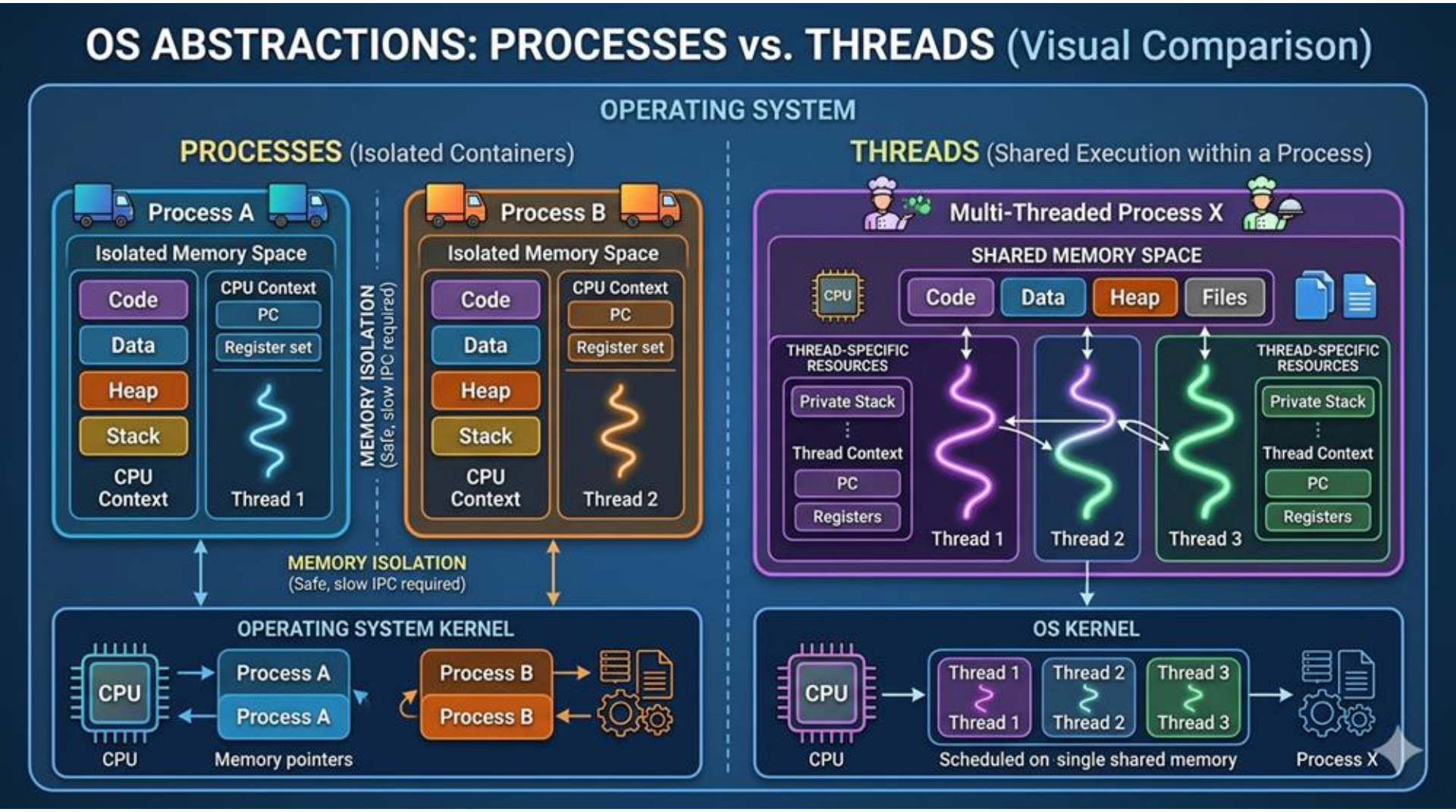
Scenario: The chef in Truck A (Process A) has a special spice that the chef in Truck B (Process B) needs. How do they get it to them? (Think about the effort: stopping the truck, driving across town, handing it over).
Scenario: In the Busy Diner (Threads), Chef 1 needs the spice that is currently in the shared pantry. How hard is it for them to get it?

The Question: If we want to work as fast as possible and share data constantly, which model is more efficient? What is the 'cost' of the Process Model's safety?

Scenario: In the Busy Diner, two chefs both reach for the *last* egg in the fridge at the exact same microsecond.

The Question: In a multi-threaded program, what happens if two threads try to change the same piece of data at the exact same time? How do we prevent 'Kitchen Chaos' ?

Processes Vs. Threads



Processes Vs. Threads

Feature	Process	Thread
Definition	An independent execution unit (a "running program").	A lightweight unit of execution within a process.
Memory Space	**Isolated.** Each process has its own address space (Code, Data, Heap, Stack).	**Shared.** All threads in a process share the same Code, Data, and Heap.
Resource Usage	High overhead. Requires more memory and system resources.	Low overhead. Uses much less memory as it shares process resources.
Context Switching	**Slower.** The OS must save/load memory maps and hardware registers.	**Faster.** Only needs to save/load registers and stack pointers; no memory switch.
Communication	Difficult/Slow. Requires **IPC** (Inter-Process Communication like pipes or sockets).	Easy/Fast. Threads communicate directly by reading/writing shared memory.
Isolation/Safety	**High.** If one process crashes (e.g., a "Segment Fault"), others are unaffected.	**Low.** If one thread crashes, it often brings down the entire process and all other threads.
Creation	Time-consuming to create and terminate.	Very quick to spawn or kill.

Threading And Synchronization - 1

Assume threads in a program that loops over adding 1 to a number.

Time	Thread A (The "Appetizer" Chef)	Thread B (The "Soup" Chef)	Shared Counter Value
T1	Reads Counter (**10**)	—	10
T2	—	Reads Counter (**10**)	10
T3	Calculates 10 + 1	—	10
T4	**Writes 11 to memory**	Calculates 10 + 1	**11**
T5	—	**Writes 11 to memory**	**11** (Error!)

Threading And Synchronization - 2

Assume threads in a program that loops over adding 1 to a number.

Time	Thread A	Thread B	Shared Counter Value
T1	**Acquires Lock** (Takes Key)	—	10
T2	Reads Counter (10)	Tries to Acquire Lock...	10
T3	Writes **11** to memory	**Blocked** (Waiting in line)	11
T4	**Releases Lock** (Returns Key)	—	11
T5	—	**Acquires Lock** (Takes Key)	11
T6	—	Reads Counter (**11**)	11
T7	—	Writes **12** to memory	**12** (✅ Success!)

Python And Threading

- Does Python support threading?

- Is Threading useful in Python?
  - The “GIL” bottleneck in Python

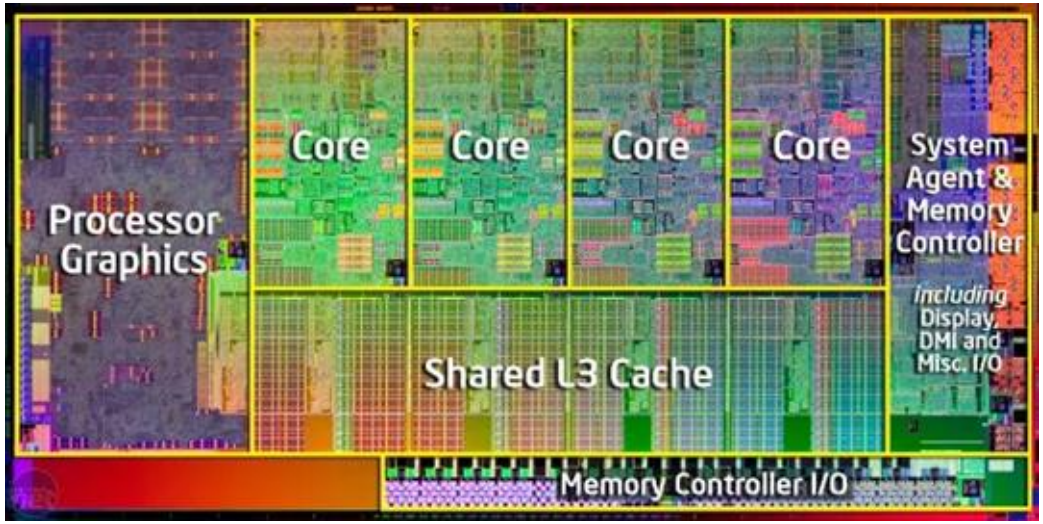
    GIL (Global Interpreter Lock) ensures only one thread executes Python bytecode at a time.

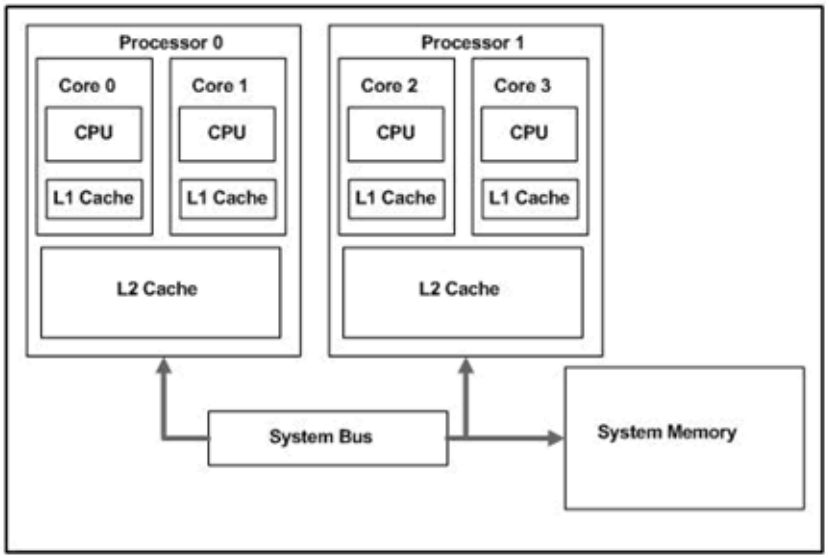
    Note 1: Python 3.12 introduced per-Interpreter GILs, allowing separate sub-interpreters to run on different cores simultaneously.

    Note 2: Python 3.13 allows developers to run Python without the GIL entirely!
  - CPU Bound tasks
  - Mixture of I/O and CPU bound tasks.

Multiprocessor

- Modern computers often have multiple processors and multiple *cores* per processor
- **Concurrency:** Multiple processors/cores run different/same set of instructions simultaneously on different/*shared* data
- New levels of shared caches are added

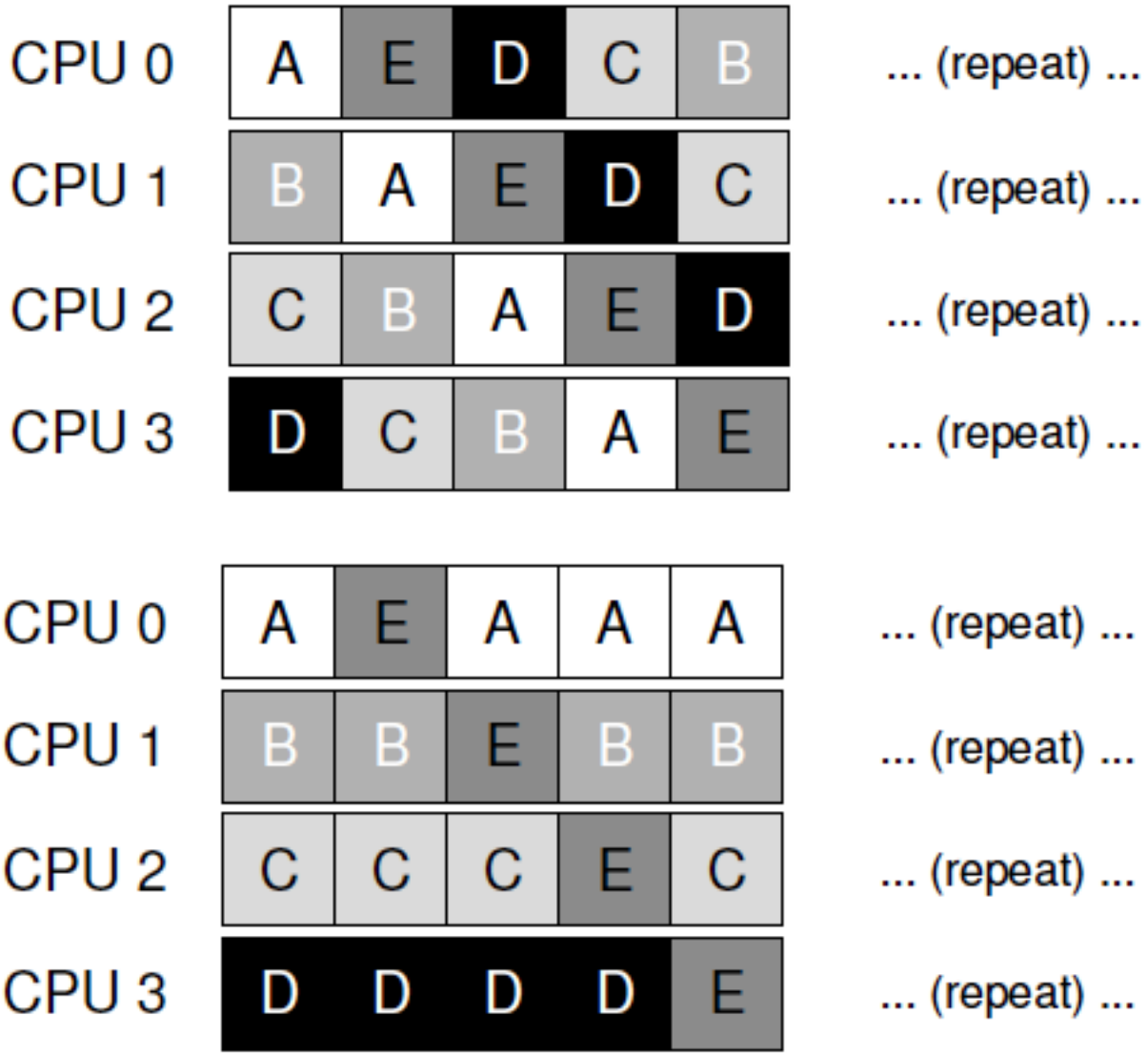




Multiprocessor Scheduling

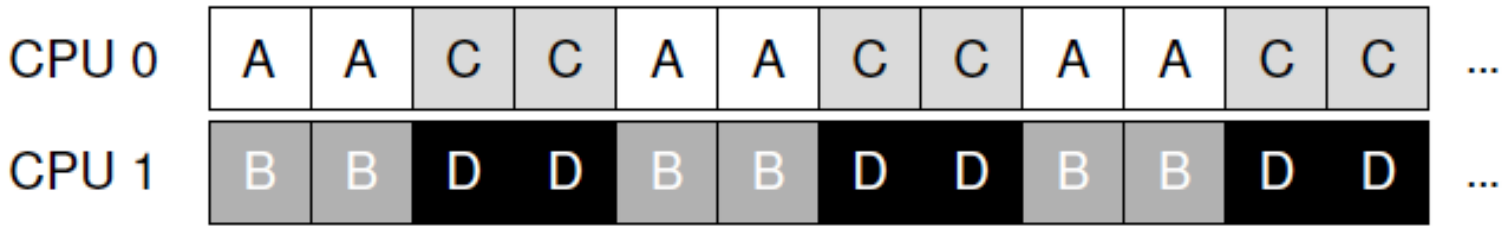
- Scheduling for multiprocessing/multicore is more complex
- **Load Balancing:** Ensuring that different cores/processors are kept roughly equally busy, i.e., reduce **idle times**
- Single- (SQMS) vs. Multi-queue multiprocessor scheduling (MQMS)
  - One queue vs. each processor/core having its own job queue

SQMS Options

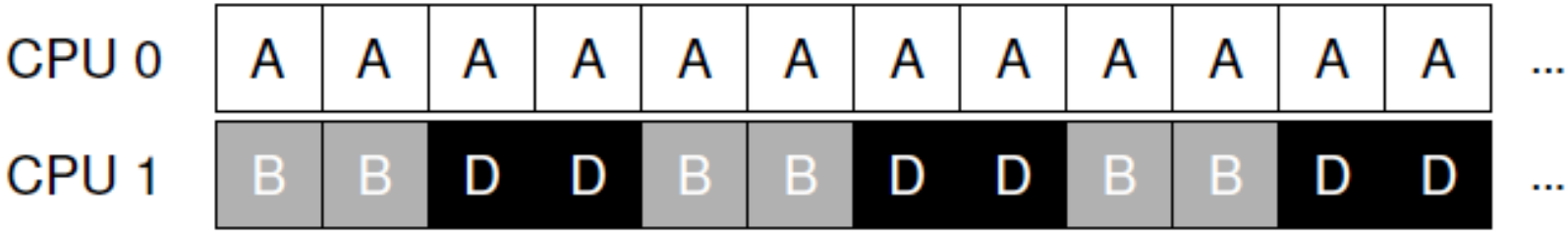


SQMS Options

RR on each queue with Load Balancing



After C finishes and we continue RR on each queue.



MQMS – Analysis

Working with
- load balancing,
- fairness,
- cache affinity,
- and other factors

make this a hard problem to solve – no perfect solution yet!

Your chance at fame and fortune:

Use ML techniques to build an adaptive OS MP scheduler!